一章作业：

1．C++是何时产生的？它和 C 语言之间有何关系？

美国 AT&T 贝尔实验室的本贾尼·斯特劳斯特卢普（Bjarne Stroustrup）博士在 20 世纪 80 年代初期发明并实现了 C++。

一开始 C++是作为 C 语言的增强版出现的，从给 C 语言增加类开始，不断的增加新特性。虚函数（virtual function）、运算符重载（operator overloading）、多重继承（multiple inheritance）、模板（template）、异常（exception）、RTTI、命名空间（name space）逐渐被加入标准。

3．面向对象的核心思想有哪些？

1.封装性，对象可以对其内部资源的可见性进行限定，每一个对象都有一个接口，决定了其他对象如何与之相互作用。
2.多态性，指相同名称的方法在不同的类中呈现不同的行为。
3.继承，可以从一个已存在的类的基础上建立新的派生类，派生类可以继承覆盖原有的类的属性和方法或是添加新的类成员。

4．相对于传统的 C 语言，C++引入哪些新的特性？《注意与前面题的比较》

类和对象、继承、多态、虚函数和 RTT1（运行阶段类型识别）、函数重载、引用变量
泛型（独立于类型的）编程，这种技术是由模版和标准模版库（STL）提供的
处理错误条件的异常机制、管理函数、类和变量名的名称空间

5．面向对象编程语言有哪些主要特点？《与“00 方法”略有不同，要谈一下“语言”》

1、维护简单
模块化是面向对象编程中的一个特征。实体被表示为类和同一名字空间中具有相同功能的类，我们可以在名字空间中添加一个类而不会影响该名字空间的其他成员。
2、可扩充性
面向对象编程从本质上支持扩充性。如果有一个具有某种功能的类，就可以很快地扩充这个类，创建一个具有扩充的功能的类。
3、代码重用
由于功能是被封装在类中的，并且类是作为一个独立实体而存在的，提供一个类库就非常简单，我们可以通过提供符合需求的类来扩充这些功能。

二章作业：

1 c++与 c 中 const 的区别？

c++中用 const 定义了一个常量后，不会分配一个空间给它，而是将其写入符号表，c++比 c 好的地方就在于使用了 constant folding 的机制，使得常量的值跟对应的内存空间无关，从而保护了该常量值，通过 int*p = (int*)(&a);这种方法，可以直接修改 const 常量对应的内存空间中的值，但是这种修改不会影响到常量本身的值，因为用到 a 的时候，编译器根

本不会去进行内存空间的读取。这就是 c++的常量折叠（constant folding），即将 const 常量放在符号表中，而并不给其分配内存。
c 中不是这样，.c 没有 constant folding 的概念，用 const 定义一个常量的时候，编译器会直接开辟一个内存空间存放该常量。.c 和.cpp,观察输出的 i 和*p 都是不相同的。

2 引用的实现机理？优点

引用不是定义一个新的变量，而是给一个已经定义的变量重新起一个别名。一个变量可取多个别名；引用必须初始化；引用只能在初始化的时候引用一次，不能改变为再引用其他的变量。指针比引用更灵活，但是也更危险。使用指针时一定要注意检查指针是否为空。指针所指的地址释放以后最好置 NULL，否则可能存在野指针。

第三章

1. 用引用传递实现交换变量函数 void mySwap(int &a,int &b)

```
void mySwap(int &a,int &b)
{
    int temp;
    temp=a;
    a=b;
    b=temp;
}
```

2. 同名的重载函数之间对形参有何要求？

如果两个函数声明的返回类型和形参表完全匹配，则将第二个函数声明视为第一个的重复声明。如果两个函数的形参表完全相同，但返回类型不同，则第二个声明是错误的，函数不能仅仅基于不同的返回类型而实现重载。

形参与 const 形参的等价性仅适用于非引用形参。有 const 引用形参的函数与有非 const 引用形参的函数是不同的。类似地，如果函数带有指向 const 类型的指针形参，则与带有指向相同类型的非 const 对象的指针形参的函数不相同。

函数重载：函数的名字相同，但是形参列表不同。重载函数可以在一定程度上减轻程序员起名字、记名字的负担。main 函数不能重载。函数具有相同的名字，调用函数时编译器会根据实参的类型确定调用哪一个函数。不允许两个函数除了返回类型不同外，其他要素都相同。

四章作业：
1.什么是构造函数、析构函数？

构造函数可自动完成对象的初始化任务，全局对象和静态对象的构造函数在 main()函数执行之前就被调用，局部静态对象的构造函数是当程序第一次执行到相应语句时才被调用。然而给出一个外部对象的引用性声明时，并不调用相应的构造函数，因为这个外部对象只是引用在其他地方声明的对象，并没有真正地创建一个对象。

析构函数(destructor) 与构造函数相反，当对象结束其生命周期，如对象所在的函数已调用完毕时，系统自动执行析构函数。析构函数往往用来做“清理善后” 的工作。析构函数名也应与类名相同，只是在函数名前面加一个位取反符~，例如~stud( )，以区别于构造函数。它不能带任何参数，也没有返回值（包括 void 类型）。只能有一个析构函数，不能重载。如果用户没有编写析构函数，编译系统会自动生成一个缺省的析构函数（即使自定义了析构函数，编译器也总是会为我们合成一个析构函数，并且如果自定义了析构函数，编译器在执行时会先调用自定义的析构函数再调用合成的析构函数），它也不进行任何操作。所以许多简单的类中没有用显式的析构函数。

2.缺省构造函数有几种写法？改成几种形式？否则同学答两种，因为（1）不用“写”

在 C++的一个类中，如果构造函数没有参数，或者构造函数的所有参数都有默认值，就可以称其为缺省构造函数。一个类中，只能有一个缺省构造函数。
①构造函数 Class()没有参数，就是 MyClass 的缺省（默认）构造函数。
②构造函数 Class(int i=10)的所有参数（此处只有一个参数 i）都有默认值，这样的形式。

3.何谓拷贝构造函数?系统自动调用拷贝构造函数的三种情况。

拷贝构造函数，它只有一个参数，参数类型是本类的引用。拷贝构造函数的参数可以是 const 引用，也可以是非 const 引用。一般使用前者，这样既能以常量对象（初始化后值不能改变的对象）作为参数，也能以非常量对象作为参数去初始化其他对象。一个类中可以写两个拷贝构造函数，一个的参数是 const 引用，另一个的参数是非 const 引用。

1.使用类的一个对象初始化另一个对象
2.当函数的形参是类的对象时（也就是值传递时），如果是引用传递则不会调用
3.函数返回值为类对象，创建临时对象作为返回值，返回值为引用不会调用拷贝构造函数

4.组合类带参构造函数的调用过程。

构造函数调用条件：在对象被创建时自动调用
拷贝构造函数调用条件：
1.当用类的一个对象去初始化该类的另一个对象时
2.如果函数的形参是类的对象，调用函数时，进行实参和形参结合时
3.如果函数的返回值是类的对象，函数执行完成返回调用者时

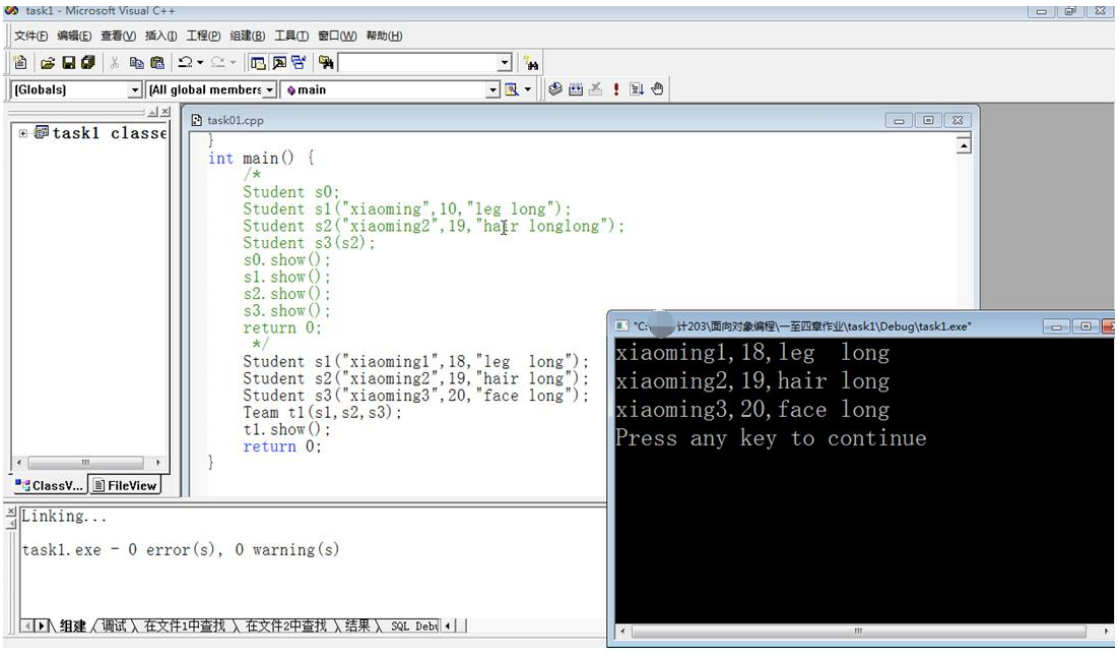
构造函数调用顺序如下：
1.调用内嵌对象的构造函数，调用顺序按照内嵌对象在组合类的定义中出现的次序。内嵌对象在构造函数的初始化列表中出现的顺序与内嵌对象构造函数的调用顺序无关。
2.执行本类的构造函数函数体。
（析构函数的调用执行顺序与构造函数刚好相反）

5. 定义一个 Student 类，数据成员有长度为 20 的字符数组 name、整数型年龄 age 和字符指针类型变量 resume（注意：必须是指针），用于保存姓名、年龄和简历。函数成员包括缺省形式构造函数、带参构造函数用于初始化数据成员，析构函数释放空间。

函数 show 显示姓名、年龄、简历。定义学习小组 Team 类,包含 3 个学生，显示成员信息函数 show()

（第七章学完再完善，从 Student 派生出大学生类 CollegeStudent，增加长整形数据成员 number 用于保存学号，函数成员包括带参构造函数用于初始化数据成员，函数 show 显示大学生全部信息。）

```
#include <iostream>
#include <string>
using namespace std;
class Student
{
private:
    char name[20];
    int age;
    char *resume;
public:
    Student();
    Student(char *Name, int Age, char *Resume);
    Student(Student & AnotherStu);
    ~Student();
    void show();
};
Student::Student(){//构造
    name[0]='\0';
    age=18;
    resume=new char[1];
    resume[0]='\0';
}
Student::Student(char *Name, int Age, char
*Resume){//带参构造
    strcpy(name,Name);
    age=Age;
    if(Resume==NULL){
        resume=new char[1];
        resume[0]='\0';
    }
    else{
        resume=new char[strlen(Resume)+1];
        strcpy(resume, Resume);
    }
}
Student::Student(Student &AnotherStu){//拷贝
构造
    strcpy(name, AnotherStu.name);
    age=AnotherStu.age;
    resume=new
char[strlen(AnotherStu.resume)+1];
    strcpy(resume, AnotherStu.resume);
}
Student::~Student(){//析构
    if(resume!=NULL)
        delete [] resume;
}
void Student::show(){
   cout<<name<<","<<age<<","<<resume<<endl;
}
class Team
{
private:
    Student s1,s2,s3;
public:
    Team();
    Team(Student S1,Student S2,Student S3);
    ~Team();//析构
    void show();
};
Team::Team(){}//构造
Team::Team(Student S1,Student S2,Student
S3):s1(S1),s2(S2),s3(S3){}//拷贝
Team::~Team(){}//析构
void Team::show(){//显示
    s1.show();
    s2.show();
    s3.show();
}
int main() {
    Student s1("xiaoming1",18,"leg  long");
    Student s2("xiaoming2",19,"hair long");
    Student s3("xiaoming3",20,"face long");
    Team t1(s1,s2,s3);
    t1.show();
    return 0;
}
```


task1 - Microsoft Visual C++
task01.cpp
int main() {
/*
Student s0;
Student s1("xiaoming",10,"leg long");
Student s2("xiaoming2",19,"hair longlong");
Student s3(s2);
s0.show();
s1.show();
s2.show();
s3.show();
return 0;
*/
Student s1("xiaoming1",18,"leg long");
Student s2("xiaoming2",19,"hair long");
Student s3("xiaoming3",20,"face long");
Team t1(s1,s2,s3);
t1.show();
return 0;
}
Linking...
task1.exe - 0 error(s), 0 warning(s)
xiaoming1,18,leg long
xiaoming2,19,hair long
xiaoming3,20,face long
Press any key to continue